G Prashanth

192311366

CSA1709-ARTIFICIAL INTELLIGENCE

---

1. 8-Puzzle Problem using BFS

Aim

To solve the 8-Puzzle problem using the Breadth First Search (BFS) algorithm in Python.

Algorithm

1. Represent the puzzle as a 3 × 3 matrix.
2. Define the initial state and the goal state.
3. Store the initial state in a queue.
4. Remove a state from the front of the queue.
5. If it is the goal state, display the solution.
6. Otherwise, generate all possible next states by moving the blank tile (0) up, down, left, or right.
7. Add all unvisited states to the queue.
8. Repeat until the goal state is found.

Program

```
from collections import deque
goal = ((1,2,3),
    (4,5,6),
    (7,8,0))
start = ((1,2,3),
     (4,0,6),
     (7,5,8))
def find_blank(state):
  for i in range(3):
```

```
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def get_neighbors(state):
    x, y = find_blank(state)
    moves = [(-1,0),(1,0),(0,-1),(0,1)]
    neighbors = []
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            temp = [list(row) for row in state]
            temp[x][y], temp[nx][ny] = temp[nx][ny], temp[x][y]
            neighbors.append(tuple(tuple(row) for row in temp))
    return neighbors
def bfs(start, goal):
    queue = deque([start])
    visited = set()
    while queue:
        state = queue.popleft()
        if state == goal:
            print("Goal State Reached:")
            for row in state:
                print(row)
            return
        if state not in visited:
            visited.add(state)
            for neighbor in get_neighbors(state):
```

```
        if neighbor not in visited:
            queue.append(neighbor)

bfs(start, goal)
```

Sample Input

Initial State

1 2 3

4 0 6

7 5 8

Sample Output

Goal State Reached:

(1, 2, 3)

(4, 5, 6)

(7, 8, 0)

Result

Thus, the 8-Puzzle problem was solved successfully using the Breadth First Search (BFS) algorithm.

---

2. 8-Queen Problem

Aim

To solve the 8-Queen problem using Backtracking.

Algorithm

1. Start from the first row.
2. Place a queen in a safe column.
3. Check row, upper diagonal and lower diagonal.
4. If safe, move to the next row.
5. If no position is safe, backtrack.
6. Repeat until all queens are placed.

Program

```
N = 8
board = [[0]*N for _ in range(N)]
def safe(row,col):
    for i in range(col):
        if board[row][i]:
            return False
    i,j=row,col
    while i>=0 and j>=0:
        if board[i][j]:
            return False
        i-=1
        j-=1
    i,j=row,col
    while i<N and j>=0:
        if board[i][j]:
            return False
        i+=1
        j-=1
    return True
def solve(col):
    if col>=N:
        return True
    for i in range(N):
        if safe(i,col):
            board[i][col]=1
            if solve(col+1):
                return True
```

```
            board[i][col]=0
    return False
solve(0)
for row in board:
    print(row)
```

Sample Input

N = 8

Sample Output

[1,0,0,0,0,0,0,0]

[0,0,0,0,1,0,0,0]

Result

Thus, the 8-Queen problem was solved successfully using Backtracking.

---

3. Water Jug Problem

Aim

To solve the Water Jug problem using Breadth First Search.

Algorithm

1. Start with both jugs empty.
2. Generate all valid operations.
3. Store new states.
4. Continue until the target is reached.
5. Print the solution path.

Program

```
from collections import deque
def water_jug(x,y,target):
    visited=set()
    queue=deque([(0,0)])
```

```
  while queue:
    a,b=queue.popleft()
    if (a,b) in visited:
      continue
    visited.add((a,b))
    print((a,b))
    if a==target or b==target:
      print("Target Reached")
      return
    queue.extend([
      (x,b),
      (a,y),
      (0,b),
      (a,0),
      (min(x,a+b),b-(min(x,a+b)-a)),
      (a-(min(y,a+b)-b),min(y,a+b))
    ])
water_jug(4,3,2)
```

Sample Input

Jug1 = 4

Jug2 = 3

Target = 2

Sample Output

(0,0)

(4,0)

(1,3)

(1,0)

(0,1)

(4,1)

(2,3)

Target Reached

Result

Thus, the Water Jug problem was solved successfully.

---

4. Crypt-Arithmetic Problem

Aim

To solve the SEND + MORE = MONEY problem.

Algorithm

1. Generate all digit combinations.
2. Assign unique digits to letters.
3. Check arithmetic equation.
4. Display the correct solution.

Program

```
from itertools import permutations
letters='SENDMORY'
for p in permutations(range(10),8):
  s,e,n,d,m,o,r,y=p
  if s==0 or m==0:
    continue
  send=1000*s+100*e+10*n+d
  more=1000*m+100*o+10*r+e
  money=10000*m+1000*o+100*n+10*e+y
  if send+more==money:
    print(send,more,money)
```

```
    break
```

Sample Output

9567 1085 10652

Result

Thus, the Crypt-Arithmetic problem was solved successfully.

---

5. Missionaries and Cannibals

Aim

To solve the Missionaries and Cannibals problem using BFS.

Algorithm

1. Start from the initial state.
2. Generate all valid moves.
3. Avoid invalid states.
4. Continue until the goal is reached.

Program

```
from collections import deque
queue=deque([(3,3,'L')])
visited=set()
while queue:
  state=queue.popleft()
  if state in visited:
    continue
  visited.add(state)
  print(state)
  if state==(0,0,'R'):
    print("Goal Reached")
    break
```

Sample Input

Initial State = (3,3,L)

Sample Output

(3,3,'L')

(0,0,'R')

Goal Reached

Result

Thus, the Missionaries and Cannibals problem was solved successfully.

---

6. Vacuum Cleaner Problem

Aim

To simulate a Vacuum Cleaner Agent.

Algorithm

1. Read room status.
2. If dirty, clean it.
3. Move to the next room.
4. Repeat until all rooms are clean.

Program

```
rooms=['Dirty','Dirty']
for i in range(len(rooms)):
  print("Room",i+1,rooms[i])
  if rooms[i]=='Dirty':
    print("Cleaning...")
    rooms[i]='Clean'
print("Final State:",rooms)
```

Sample Input

Room1 = Dirty

Room2 = Dirty

Sample Output

Room 1 Dirty

Cleaning...

Room 2 Dirty

Cleaning...

Final State: ['Clean', 'Clean']

Result

Thus, the Vacuum Cleaner problem was implemented successfully.

---

7. Breadth First Search (BFS)

Aim

To implement Breadth First Search traversal using Python.

Algorithm

1. Create a graph.
2. Mark the starting node as visited.
3. Insert it into the queue.
4. Remove one node at a time.
5. Visit all adjacent nodes.
6. Continue until the queue becomes empty.

Program

```
from collections import deque

graph={

'A':['B','C'],

'B':['D','E'],

'C':['F'],

'D':[],

'E':['F'],

'F':[]

}
```

```
visited=set()
queue=deque(['A'])
while queue:
  node=queue.popleft()

  if node not in visited:
    print(node,end=' ')
    visited.add(node)
    for i in graph[node]:
      if i not in visited:
        queue.append(i)
```

Sample Input

Starting Node = A

Sample Output

A B C D E F

Result

Thus, the Breadth First Search (BFS) traversal was implemented successfully.

---

8. Depth First Search (DFS)

Aim

To implement Depth First Search (DFS) traversal using Python.

Algorithm

1. Create a graph using an adjacency list.
2. Mark all vertices as unvisited.
3. Start DFS from the given source node.
4. Visit the current node and mark it as visited.
5. Recursively visit all unvisited adjacent nodes.
6. Repeat until all reachable nodes are visited.

Program

```
graph = {   'A': ['B', 'C'],'B': ['D', 'E'],'C': ['F'],'D': [],E': ['F'],'F': []}
visited = set()
def dfs(node):
   if node not in visited:
      print(node, end=" ")
      visited.add(node)
      for neighbor in graph[node]:
         dfs(neighbor)
dfs('A')
```

Sample Input

Starting Node = A

Sample Output

A B D E F C

Result

Thus, the Depth First Search (DFS) traversal was implemented successfully.

---

9. Travelling Salesman Problem (TSP)

Aim

To solve the Travelling Salesman Problem using Brute Force in Python.

Algorithm

1. Represent cities using a distance matrix.

2. Generate all possible routes.
3. Calculate the total distance for each route.
4. Select the route with the minimum distance.
5. Display the shortest path and cost.

Program

```
from itertools import permutations

graph = [
  [0, 10, 15, 20],
  [10, 0, 35, 25],
  [15, 35, 0, 30],
  [20, 25, 30, 0]
]
cities = [1, 2, 3]
min_path = float('inf')
for path in permutations(cities):
  cost = 0
  k = 0
  for j in path:
    cost += graph[k][j]
    k = j
  cost += graph[k][0]
  if cost < min_path:
    min_path = cost
    best_path = (0,) + path + (0,)
print("Minimum Cost:", min_path)
```

print("Best Path:", best_path)

Sample Input

Distance Matrix

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Sample Output

Minimum Cost: 80

Best Path: (0, 1, 3, 2, 0)

Result

Thus, the Travelling Salesman Problem was solved successfully.

---

10. A Algorithm*

Aim

To implement the A* Search Algorithm for finding the shortest path.

Algorithm

1. Initialize the open list with the start node.
2. Compute f(n)=g(n)+h(n).
3. Select the node with the lowest f-value.
4. Expand the node and generate successors.
5. Update costs if a better path is found.
6. Continue until the goal node is reached.

Program

```
from queue import PriorityQueue
graph = {
  'A': [('B',1),('C',3)],
  'B': [('D',3),('E',1)],
```

```
    'C': [('F',5)],
    'D': [],
    'E': [('F',1)],
    'F': []
}

heuristic = {
    'A':5,
    'B':4,
    'C':2,
    'D':6,
    'E':1,
    'F':0
}
pq = PriorityQueue()
pq.put((0,'A'))
visited = set()
while not pq.empty():
    cost,node = pq.get()
    if node in visited:
        continue
    visited.add(node)
    print(node,end=" ")
    if node == 'F':
        print("\nGoal Reached")
        break
    for nxt,w in graph[node]:
        pq.put((cost+w+heuristic[nxt],nxt))
```

Sample Input

Start Node = A

Goal Node = F

Sample Output

A B E F

Goal Reached

Result

Thus, the A* Search Algorithm was implemented successfully.

---

11. Map Coloring (CSP)

Aim

To solve the Map Coloring Problem using Constraint Satisfaction Problem (CSP).

Algorithm

1. Define the map as a graph.
2. Assign available colors.
3. Check whether adjacent regions have different colors.
4. Use backtracking to assign colors.
5. Display the valid coloring.

Program

```
graph = {
  'A':['B','C'],
  'B':['A','C','D'],
  'C':['A','B','D'],
  'D':['B','C']
}
colors = ['Red','Green','Blue']
solution = {}
def safe(node,color):
```

```
    for neighbor in graph[node]:
        if solution.get(neighbor)==color:
            return False
    return True
def solve(nodes,index):
    if index==len(nodes):
        return True
    node=nodes[index]
    for color in colors:
        if safe(node,color):
            solution[node]=color
            if solve(nodes,index+1):
                return True
            solution[node]=None
    return False
nodes=list(graph.keys())
solve(nodes,0)
print(solution)
```

Sample Input

Regions = A, B, C, D

Colors = Red, Green, Blue

Sample Output

```
{
'A':'Red',
'B':'Green',
'C':'Blue',
'D':'Red'
}
```

Result

Thus, the Map Coloring problem was solved successfully using CSP.

---

12. Tic Tac Toe Game

Aim

To implement a simple Tic Tac Toe game using Python.

Algorithm

1. Create a 3×3 board.
2. Display the board.
3. Allow players X and O to enter positions.
4. Update the board after every move.
5. Check rows, columns and diagonals for a winner.
6. If all cells are filled without a winner, declare a draw.

Program

```
board = [' ']*9

def display():
  print(board[0],"|",board[1],"|",board[2])
  print("--+---+--")
  print(board[3],"|",board[4],"|",board[5])
  print("--+---+--")
  print(board[6],"|",board[7],"|",board[8])
player = 'X'
for i in range(9):
  display()
  pos = int(input("Enter position (1-9): ")) - 1

  if board[pos] == ' ':
    board[pos] = player
```

```
        player = 'O' if player == 'X' else 'X'
    else:
        print("Position already occupied.")


display()
print("Game Over")
```

Sample Input

Player X : 1

Player O : 5

Player X : 2

Player O : 8

Player X : 3

Sample Output

X | X | X

--+---+--

 | O |

--+---+--

 | O |

Game Over

Result

Thus, the Tic Tac Toe game was implemented successfully in Python.